

The CKA Field Guide

A pass-focused study guide built from the 74 hands-on labs in this repository — commands, playbooks, traps and a drill plan.

74

PRACTICE LABS

5

EXAM DOMAINS

66%

PASS MARK

120

MINUTES

CONTENTS

What is in this guide

This is not a Kubernetes textbook. It is a **practice manual**. Everything here is tied to a lab in this repo that sets up broken cluster state, asks you to fix it, and then grades you automatically. Read a chapter, run its labs, then move on.

- **Part 1 — The exam, and how to beat it**

Format, domain weights, the time budget, and the scoring maths that decides your result

- **Part 2 — Your first 60 seconds**

Aliases, vim setup, dry-run generators, and the docs-navigation habit

- **Part 3 — Cluster Architecture, Installation & Configuration (25%)**

RBAC, kubeadm upgrades, etcd backup/restore, certificates, static pods, Helm, Kustomize

- **Part 4 — Workloads & Scheduling (15%)**

Deployments, resources, HPA, taints, affinity, priority, manual scheduling

- **Part 5 — Storage (10%)**

StorageClasses, PV/PVC binding rules, volume modes, StatefulSet volume templates

- **Part 6 — Services & Networking (20%)**

Services, Ingress, Gateway API, NetworkPolicy, CoreDNS, CNI

- **Part 7 — Troubleshooting (30%) — the domain that decides your result**

Five playbooks that cover almost every broken-cluster question

- **Part 8 — The command arsenal**

One page you should be able to reproduce from memory

- **Part 9 — Twenty traps these labs were built to punish**

The specific mistakes that cost marks, harvested from the lab solutions

- **Part 10 — Lab index and study plan**

All 74 labs mapped to domains, plus a six-week schedule and mock-exam protocol

HOW TO RUN THE LABS

```
# Provision one lab, read the task, work it, then grade yourself
scripts/run-question.sh 56
scripts/validate-question.sh 56          # prints PASS/FAIL per check + Results: X/Y passed
scripts/cleanup-question.sh 56

# Simulate the real thing: 20 random labs, one 120-minute clock, weighted score
scripts/exam-mode.sh start -n 20 -t 120 --safe
scripts/exam-mode.sh status              # time remaining
scripts/exam-mode.sh grade               # weighted % against the 66% pass mark
scripts/exam-mode.sh cleanup
```

Scoring model: a lab is worth one point per `check` in its `validate.bash`, so a four-bug troubleshooting lab outweighs a one-line pod creation — exactly like the real exam. `--safe` filters out labs that need control-plane access or a second node, so you can drill on a single-node cluster.

PART 1

The exam, and how to beat it

1.1 What you are walking into

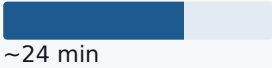
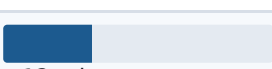
The CKA is entirely performance-based. There is no multiple choice. You get a browser-based Linux desktop with a terminal and a handful of real clusters, and a list of tasks that each require you to *change cluster state*. A grader re-runs against those clusters afterwards — much like the `validate.bash` scripts in this repo.

Item	What it means for you
Duration — 2 hours	Roughly 15–20 tasks. That is about 6–7 minutes per task , including reading. Anything you cannot start within 60 seconds should be flagged and skipped.
Pass mark — 66%	You can fail a third of the exam and still pass. Never chase a perfect answer. Partial credit is real: get the object created, then refine.
Multiple clusters	Every task begins with a context-switch command. Running the right answer on the wrong cluster scores zero.
Allowed documentation	kubernetes.io documentation and blog, plus the Helm docs, in one extra browser tab. Nothing else — no notes, no ChatGPT, no GitHub.
Retake	Registration normally includes one free retake. That is a safety net, not a plan.

Logistics (exact task count, environment vendor, retake terms) change between exam revisions — confirm the current details on the Linux Foundation / CNCF exam page before you book.

1.2 Domain weights — and where the marks actually are

These weights are the single most important planning input in this guide. Study time should follow them, not follow what you find interesting.

Domain	Weight	Share of a 120-min exam	Labs in this repo
Troubleshooting	30%	 ~36 min	15, 20, 21, 22, 25, 45, 56–64
Cluster Architecture, Installation & Configuration	25%	 ~30 min	2, 6, 8, 9, 23, 24, 36, 37, 41, 50, 65–73
Services & Networking	20%	 ~24 min	8, 11, 12, 13, 16, 34, 35, 38, 42, 51, 52, 58
Workloads & Scheduling	15%	 ~18 min	3, 4, 5, 7, 10, 18, 19, 29, 30, 32, 40, 43, 44, 47, 49, 53–55, 74
Storage	10%	 ~12 min	1, 14, 26, 27, 28, 33, 48, 61

THE ONE STRATEGIC CONCLUSION

Troubleshooting plus Cluster Architecture is 55% of the exam. Those are also the two domains people under-practise, because they need a real kubeadm cluster and they are uncomfortable. If you can reliably repair a dead control plane, restore etcd, and untangle RBAC, you are most of the way to 66% before you touch a single Deployment YAML. Part 3 and Part 7 of this guide are deliberately the longest.

1.3 The time budget, task by task

Work the paper in three passes. This is the single highest-leverage habit you can build, and `exam-mode.sh` exists so you can rehearse it.

Pass	Budget	What you do
Pass 1 Sweep	~45 min	Go top to bottom. Do every task you can finish in under 4 minutes — pod creation, a Role, a Service, a StorageClass. Flag anything bigger and move on immediately . Do not start an etcd restore in pass 1.
Pass 2 Heavy tasks	~55 min	Now attack the flagged high-weight items: cluster upgrade, etcd, broken control plane, multi-bug troubleshooting. You already have marks in the bank, so the clock pressure is off.
Pass 3 Verify	~20 min	Re-check every task's end state, in the right context. Did the rollout finish? Is the node uncordoned? Did you actually save the file to the exact path asked for?

THE CLASSIC WAY TO FAIL WITH KNOWLEDGE YOU HAD

Spending 25 minutes on a stubborn NetworkPolicy in the first half hour. Two easy 4-point tasks at the bottom of the paper go untouched, and you finish at 61%. The grader does not care how hard a question was — only whether the end state matches. **Skipping is a skill. Practise it in exam mode.**

1.4 Rules you should never break in the exam

- **Switch context first, every single task.** The task statement gives you the command. Copy it, run it, then read the task again.
- **Read the exact names.** Namespace, object name, port, path. Graders match strings. `pod-reader` is not `podreader`.
- **Verify the end state, not your command.** `kubectl apply` succeeding proves the YAML parsed, not that the pod runs. Always follow with a `get` or `rollout status`.
- **Never delete something you were told to fix.** Several labs here (61, 70, 74) explicitly grade you on *not* deleting the object.
- **Leave the cluster tidy.** Uncordon what you drained, unless told otherwise. Question 24 loses marks precisely there.

PART 2

Your first 60 seconds

Typing speed is worth real marks. Set this up the moment the exam starts — it pays for itself within three tasks.

2.1 The setup block

```
# Alias + completion. 'k' saves you ~7 keystrokes per command, hundreds over the exam.
alias k=kubectl
complete -o default -F __start_kubectl k
export do="--dry-run=client -o yaml"      # then: k run x --image=nginx $do > x.yaml
export now="--force --grace-period=0"     # instant pod deletion

# Stop retyping -n. Do this at the START of every namespaced task.
kubectl config set-context --current --namespace=<ns>

# YAML-friendly vim. Without this, every pasted manifest is a staircase.
cat <<'EOF' > ~/.vimrc
set expandtab tabstop=2 shiftwidth=2 number
set paste
EOF
```

THE GENERATOR HABIT

Almost nothing should be typed from scratch. Generate a skeleton, then edit it. This is faster *and* less error-prone than copying from the docs, because the API version is always right.

```
k run nginx --image=nginx $do > pod.yaml
k create deployment web --image=nginx --replicas=3 $do > deploy.yaml
k create job j --image=busybox $do -- /bin/sh -c 'echo hi'
k create cronjob c --image=busybox --schedule='*/1 * * * *' $do -- date
k expose deployment web --port=80 --target-port=8080 --type=NodePort $do > svc.yaml
k create role pod-reader --verb=get,list,watch --resource=pods,configmaps $do
k create rolebinding rb --role=pod-reader --serviceaccount=ns:sa $do
k create clusterrole cr --verb=list --resource=pods $do
k create configmap cm --from-literal=KEY=value $do
k create secret generic s --from-literal=PASS=hunter2 $do
k create ingress ing --rule="example.org/echo=echo-service:8080" $do
k create priorityclass high --value=999 $do
k create quota q --hard=cpu=2,memory=2Gi,pods=5 $do
k create serviceaccount app-sa $do
```

2.2 When there is no generator: `kubectl explain`

Roughly a third of exam objects (PV, StorageClass, NetworkPolicy, HPA behaviour, StatefulSet volume templates) have no `kubectl create` generator. You have two legitimate sources: the docs tab, and `explain`. `explain` is usually faster.

```
k explain pod.spec.containers.resources
k explain networkpolicy.spec.ingress --recursive      # whole subtree, field names only
k explain statefulset.spec.volumeClaimTemplates
k api-resources | grep -i policy                      # find the kind and its apiGroup
k api-versions | grep gateway                        # confirm the right apiVersion
```

2.3 Navigate the docs by structure, not by search

Nearly every `Questions.bash` in this repo ends with a documentation path such as *Concepts → Storage → Persistent Volumes*. That is deliberate. Searching `kubernetes.io` under time pressure returns blog posts and old API versions; navigating the left-hand tree gets you to a copy-pasteable example in about 15 seconds. Learn these six landing pages by heart:

When the task is about...	Go straight to
PV, PVC, StorageClass	Concepts → Storage → Persistent Volumes
NetworkPolicy	Concepts → Services, Load Balancing & Networking → Network Policies
RBAC	Reference → Access Control → Using RBAC Authorization
Probes, resources, security context	Tasks → Configure Pods and Containers
etcd, upgrades, certs, drain	Tasks → Administer a Cluster
Anything broken	Tasks → Monitoring, Logging and Debugging

PRACTISE THIS IN THE LABS

When you work a lab in this repo, do it with only the `kubernetes.io` tab open — no Google, no LLM, no this guide. The labs list the doc path so you can check afterwards whether you navigated to the same place. That habit is worth more on exam day than memorising YAML.

Cluster Architecture, Installation & Configuration

The heaviest single-topic domain, and the one that rewards rehearsal most: these tasks are long, procedural, and unforgiving of order-of-operations mistakes. Every one of them can be practised end-to-end here.

3.1 RBAC — the mental model that makes it easy

Two axes, and only two. Get these straight and RBAC stops being guesswork:

	Role (namespaced)	ClusterRole (cluster-scoped)
RoleBinding (namespaced)	Permissions apply in that one namespace.	The trick most people miss: the verbs apply <i>only</i> in the RoleBinding's namespace. One ClusterRole, reused in many namespaces, with no cluster-wide grant.
ClusterRoleBinding	Not valid.	Permissions apply in every namespace and to cluster-scoped resources.

A CLUSTERROLE DEFINES WHAT. THE BINDING DECIDES WHERE.

Lab 23 is built on exactly this: a ServiceAccount in `finance` must also list pods in `hr`, without a second Role and without cluster-wide access. The answer is a ClusterRole bound with a **RoleBinding in hr**.

Reaching for `create clusterrolebinding` there grants the verb in every namespace and fails the negative check.

```
# Build it imperatively – faster and less error-prone than YAML
k create role pod-reader -n finance --verb=get,list,watch --resource=pods,configmaps
k create rolebinding report-runner-read -n finance \
  --role=pod-reader --serviceaccount=finance:report-runner

# Same ClusterRole, scoped to one namespace by the binding
k create clusterrole pod-lister --verb=list --resource=pods
k create rolebinding report-runner-hr-read -n hr \
  --clusterrole=pod-lister --serviceaccount=finance:report-runner

# VERIFY – including the negatives. This is how the grader thinks.
SA=system:serviceaccount:finance:report-runner
k auth can-i list pods      -n finance --as=$SA      # yes
k auth can-i get configmaps -n hr      --as=$SA      # no <- the mark
k auth can-i delete pods    -n finance --as=$SA      # no <- the mark
k auth can-i --list         -n finance --as=$SA      # everything this identity can do
```

THE FOUR RBAC BUGS THAT APPEAR OVER AND OVER (LAB 59 HAS THREE OF THEM)

1. **Singular resource name.** `resources: ["pod"]` silently matches nothing. It is always plural: `pods`, `deployments`, `configmaps`.
2. **Wrong apiGroup.** Pods, Services, ConfigMaps and Secrets are in the **core** group: `apiGroups: [""]`. Deployments, ReplicaSets, DaemonSets and StatefulSets are in `apps`. Getting this wrong is the #1 cause of a "correct-looking" Role that denies everything.
3. **Subject namespace.** In a RoleBinding, a ServiceAccount subject needs its *own* `namespace:` field. It does not inherit the binding's namespace.
4. **Missing verb.** `list` does not imply `get`, and neither implies `watch`. Grant all three when asked for all three.

Aggregated ClusterRoles (Lab 65)

A ClusterRole with an `aggregationRule` has **empty rules** — the controller fills them in by selecting other ClusterRoles by label. Do not write rules into the aggregate; if you do, they get overwritten.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: { name: monitoring-admin }
aggregationRule:
  clusterRoleSelectors:
    - matchLabels: { rbac.cka.local/aggregate-to-monitoring: "true" }
rules: []           # populated automatically – verify with: k get clusterrole monitoring-admin -o yaml
```

3.2 Onboarding a user: CSR → certificate → kubeconfig (Lab 66)

A five-step pipeline that appears in some form on most exams. The certificate **CN becomes the username** and each **O becomes a group**. Kubernetes has no User object — authorisation is attached to that CN by a binding.

```
# 1. Key + CSR.  CN = username, O = group
openssl genrsa -out dev-user.key 2048
openssl req -new -key dev-user.key -subj "/CN=dev-user/O=dev-team" -out dev-user.csr

# 2. Submit it. NOTE base64 -w0 – line wrapping breaks the request field.
cat <<EOF | k apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata: { name: dev-user }
spec:
  request: $(base64 -w0 < dev-user.csr)
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages: ["client auth"]
EOF

# 3. Approve and extract
k certificate approve dev-user
k get csr dev-user -o jsonpath='{.status.certificate}' | base64 -d > dev-user.crt

# 4. Authorise – namespace-scoped, using a built-in ClusterRole
k create rolebinding dev-user-edit --clusterrole=edit --user=dev-user -n dev-ns

# 5. Build a self-contained kubeconfig (--embed-certs makes it portable)
KC=dev-user.kubeconfig
SERVER=$(k config view --minify -o jsonpath='{.clusters[0].cluster.server}')
k config view --raw --minify --flatten \
  -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' | base64 -d > ca.crt
k config set-cluster kubernetes --kubeconfig=$KC --server=$SERVER \
```



```
--certificate-authority=ca.crt --embed-certs=true
k config set-credentials dev-user --kubeconfig=$KC \
  --client-certificate=dev-user.crt --client-key=dev-user.key --embed-certs=true
k config set-context dev@k8s --kubeconfig=$KC --cluster=kubernetes --user=dev-user --namespace=dev-ns
k config use-context dev@k8s --kubeconfig=$KC

k --kubeconfig=$KC get pods -n dev-ns          # works
k auth can-i list namespaces --as=dev-user     # no
```

TRAPS

Forgetting `--embed-certs=true` produces a kubeconfig full of file paths that break the moment it is moved. And a `ClusterRoleBinding` to `edit` grants cluster-wide edit — read whether the task says "in namespace X".

3.3 etcd backup and restore (Lab 73) — the highest-value 8 minutes you can rehearse

Almost guaranteed to appear. It is pure procedure, and the marks are all-or-nothing. Rehearse it until you can do it without the docs.

```
# STEP 0 — never memorise the paths. Read them off the static pod manifest.
grep -E "listen-client-urls|trusted-ca-file|cert-file|key-file|data-dir" \
  /etc/kubernetes/manifests/etcd.yaml

# STEP 1 — snapshot
export ETCDCTL_API=3
etcdctl snapshot save /opt/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# STEP 2 — verify (snapshot status needs NO certs; it reads the file)
etcdctl snapshot status /opt/etcd-backup.db --write-out=table

# STEP 3 — restore into a NEW directory. Never restore over the live data dir.
etcdctl snapshot restore /opt/etcd-backup.db --data-dir=/var/lib/etcd-restore

# STEP 4 — point the static pod at the restored dir.
#       Change the hostPath on the HOST side only; the mountPath stays.
vim /etc/kubernetes/manifests/etcd.yaml
#   volumes:
#     - name: etcd-data
#       hostPath:
#         path: /var/lib/etcd-restore    <- was /var/lib/etcd

# The kubelet restarts etcd on manifest change. Force it if slow:
mv /etc/kubernetes/manifests/etcd.yaml /tmp/ && sleep 10 && mv /tmp/etcd.yaml /etc/kubernetes/manifests/
crictl ps | grep etcd

# STEP 5 — verify: pre-backup data is back, post-backup data is gone
k get nodes
k get cm -n <ns>
```

EXPECT A SCARY GAP

While etcd restarts, `kubectll` returns connection refused for 30-90 seconds. That is normal. Wait and re-check rather than "fixing" something that is not broken. If it never comes back, `crictl ps -a | grep etcd` then `crictl logs <id>` — usually a permissions or path typo in the manifest.

3.4 Upgrading a cluster with kubeadm (Lab 24)

Order is everything, and the last step is the one people forget.

```
kubeadm upgrade plan                                # 0. what can we go to? Pick the next PATCH, never skip a
minor.
TARGET=v1.32.2

apt-mark unhold kubeadm                            # 1. kubeadm binary FIRST
apt-get update && apt-get install -y kubeadm=${TARGET#v}-1.1
apt-mark hold kubeadm
kubeadm version                                     #   confirm before continuing

kubeadm upgrade apply $TARGET -y                    # 2. control plane (on a 2nd CP node: kubeadm upgrade node)
```

```
k drain <node> --ignore-daemonsets           # 3. drain BEFORE touching the kubelet

apt-mark unhold kubelet kubectrl           # 4. kubelet + kubectrl
apt-get install -y kubelet=${TARGET#v}-1.1 kubectrl=${TARGET#v}-1.1
apt-mark hold kubelet kubectrl
systemctl daemon-reload && systemctl restart kubelet

k uncordon <node>                           # 5. <-- THE STEP THAT LOSES MARKS
k get nodes                                 #   Ready, new version, NOT SchedulingDisabled
```

VERSION SKEW, IN ONE LINE

The kubelet may trail the API server, but must never lead it. That is the whole reason the control plane is upgraded before the kubelet — and why you upgrade one minor version at a time. On worker nodes the middle step is `kubeadm upgrade node`, not `upgrade apply`.

3.5 Certificates (Lab 68)

```
kubeadm certs check-expiration > /tmp/exam/before.txt # full overview, all certs
kubeadm certs renew apiserver                        # renew ONE cert, not 'renew all'
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates # independent proof
```

If a task says "renew only X", `kubeadm certs renew all` is a wrong answer even though the cluster still works. Read the verb, then read the scope.

3.6 Static pods (Lab 67)

A static pod is owned by the **kubelet**, not the API server. The kubelet watches `staticPodPath` (confirm with `grep staticPodPath /var/lib/kubelet/config.yaml`; on kubeadm it is `/etc/kubernetes/manifests`) and creates a read-only **mirror pod** named `<name>-<nodename>`, owned by the Node object.

THE CONSEQUENCE YOU MUST INTERNALISE

`kubectrl delete pod` on a mirror pod does nothing lasting — the kubelet recreates it in seconds. **Only removing the manifest file truly deletes it.** This is also why every control-plane repair in Part 7 is done by editing a file on disk, never with `kubectrl`.

3.7 Helm (Labs 2, 36, 72)

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm search repo bitnami/nginx --versions # list available chart versions
helm show values argocd/argo-cd --version 7.7.3 | grep -i crd -C 4 # FIND the right value

helm install web bitnami/nginx -n ns --create-namespace --set replicaCount=1
helm upgrade web bitnami/nginx -n ns --set replicaCount=3 # revision 2
helm history web -n ns
helm rollback web 1 -n ns # rolling back CREATES revision 3
helm template web bitnami/nginx --version X -n ns > /root/out.yaml # render, don't install
```

THE ARGO CD TRAP (LAB 2)

`--skip-crds` does **not** reliably suppress CRDs. Many charts ship CRDs as ordinary templates gated by a chart *value* — for `argo-cd` it is `--set crds.install=false`. The generalisable skill: when a task says "configure the chart so that X", run `helm show values <chart> | grep -i <thing>` and find the value. Do not guess a flag.

3.8 Kustomize (Labs 37, 71)

```
# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [ ../../base ]
namespace: prod-ns
namePrefix: prod-          # deployment 'api' becomes 'prod-api'
replicas:
  - { name: api, count: 3 }
commonAnnotations: { owner: platform-team }
patches:
  - path: probe-patch.yaml      # JSON6902 patch file
    target: { kind: Deployment, name: api } # target uses the ORIGINAL name, pre-prefix
secretGenerator:
  - name: app-secret
    literals: [ API_KEY=abc123 ] # generated name gets a content hash suffix

# Preview before you apply – this is the fastest way to catch a bad patch path
kubectl kustomize overlays/production/
kubectl apply -k overlays/production/
```

THREE KUSTOMIZE TRAPS

(1) `patches.target` matches the **pre-prefix** name. (2) Generated ConfigMaps/Secrets get a hash suffix — `prod-app-secret-9t2bkm74hc` — so never hard-code the generated name elsewhere. (3) Newer kustomize prefers `labels:` over the deprecated `commonLabels:`; both still apply cleanly.

3.9 CRDs (Lab 6)

```
k get crd | grep cert-manager          # list installed custom resources
k explain certificate.spec.subject      # read a CRD's schema – works for ANY installed CRD
k api-resources --api-group=cert-manager.io # kinds, shortnames, namespaced or not
```

`kubectl explain` reads the OpenAPI schema the CRD published, so it documents third-party resources exactly like built-ins. That is the intended answer whenever a task says "extract the documentation for field X" — you do not need the vendor's website.

Workloads & Scheduling

The fastest marks on the paper. Most of these tasks are 2–4 minutes with a generator and one edit — which is exactly why you must not lose them.

4.1 Resource requests and limits — the calculation task (Labs 4, 19)

A recurring exam pattern: "divide the node's resources evenly across N replicas." It looks like a maths trap; it is actually a two-command lookup plus a formula.

```
k describe node | grep -A6 "Allocatable:"           # the numerator
k describe node | grep -A10 "Allocated resources"    # what is already spoken for
```

THE FORMULA

per-pod request = (allocatable – system overhead) ÷ replicas, then **round down**.

System overhead is kube-proxy, CoreDNS and friends — budget ~150m CPU / ~250Mi on a small node, ~600m on a 3-CPU exam node. Memory in $Ki \div 1024 = Mi$. CPU: 1 core = 1000m.

Worked example — 1 CPU / 1800Mi node, 3 replicas: CPU $(1000 - 150) \div 3 \approx 283m \rightarrow$ use **250m**. Memory $(1800 - 250) \div 3 \approx 516Mi$, but the existing limit is 500Mi $\rightarrow$ use **300Mi**.

THREE THINGS THAT SINK THIS QUESTION

1. **Requests must never exceed limits.** If your computed request is above the existing limit, drop below the limit — do not raise the limit unless told to.
2. **Count containers, not pods.** Lab 4 has 3 pods $\times$ 2 co-located containers = **6** consumers. Init containers run before the main container, so they do not add to the concurrent total, but the task may still require identical values on both.
3. **Only requests drive scheduling.** Limits and actual usage are irrelevant to whether a pod is Pending.

```
# Fastest edit path – no YAML at all
k -n ns set resources deployment/wordpress --containers=wordpress \
  --requests=cpu=250m,memory=300Mi
k -n ns set resources deployment/web --limits=cpu=500m,memory=512Mi

# Or patch in place (Lab 18) – note limits and requests are separate maps
k patch deployment resource-app -n patch-ns -p '{"spec":{"template":{"spec":{"containers":
[{"name":"nginx","resources":{"limits":{"cpu":"500m","memory":"512Mi"}}}]}}}]'
```

Why a strategic-merge patch is safe on containers: the container list is merged by the `name` key, so naming the container preserves everything else about it. That is why you must always include `"name":"nginx"` — omit it and the patch is rejected or replaces the whole list.

4.2 Scheduling controls — pick the right tool

Mechanism	Direction	Use when
nodeSelector	Pod → node	Simple exact label match. Fastest to write.
nodeAffinity	Pod → node	You need operators (<code>In</code> , <code>Exists</code> , <code>NotIn</code>) or a soft preference.
Taint + toleration	Node → pod	The node <i>repels</i> pods. A toleration is permission to land, not a guarantee of landing.
podAffinity / antiAffinity	Pod → pod	Spread or co-locate relative to other pods, via <code>topologyKey</code> .
nodeName / Binding API	Bypass scheduler	Manual placement (Lab 74).

```
# Taints (Labs 10, 32)
k taint node node01 PERMISSION=granted:NoSchedule
k taint node node01 PERMISSION=granted:NoSchedule-      # trailing '-' REMOVES it
k describe node node01 | grep -i taint

# The matching toleration
tolerations:
- key: "PERMISSION"
  operator: "Equal"
  value: "granted"
  effect: "NoSchedule"

# Tolerate EVERYTHING (DaemonSet pattern, Lab 47)
tolerations:
- operator: "Exists"

# Required node affinity (Lab 30)
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: disk, operator: In, values: ["ssd"] }

# Anti-affinity: never two of these on one node (Lab 44)
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - { key: app, operator: In, values: ["demo"] }
      topologyKey: kubernetes.io/hostname
```

THE THREE EFFECTS, AND THE ONE THAT SURPRISES PEOPLE

`NoSchedule` blocks new pods. `PreferNoSchedule` is a soft hint. `NoExecute` **evicts pods that are already running**. Applying `NoExecute` to a busy node during an exam task will move workloads you did not intend to move.

4.3 Manual scheduling and the Binding API (Lab 74)

Two different problems with two different answers — and knowing which is which is the whole question.

```
# Pod does NOT exist yet -> spec.nodeName skips the scheduler; the kubelet just takes it
spec:
  nodeName: node01
  containers: [ { name: nginx, image: nginx:1.25 } ]

# Pod ALREADY exists -> nodeName is immutable. POST a Binding – literally what
# kube-scheduler does internally. This preserves the pod's UID and schedulerName.
k create -f - <<EOF
apiVersion: v1
kind: Binding
metadata: { name: orphan-pod, namespace: manual-sched }
target: { apiVersion: v1, kind: Node, name: node01 }
EOF
```

WHY THIS APPEARS ON THE EXAM

It proves you understand that scheduling is just *writing a node name into the pod object*. A pod requesting a `schedulerName` that does not exist will stay Pending forever with **no FailedScheduling event at all** — the absence of events is the diagnostic clue.

4.4 Deployments, rollouts and rollbacks (Lab 43)

```
k set image deployment/app-v1 nginx=nginx:1.20      # trigger a rollout
k rollout status deployment/app-v1                  # ALWAYS wait – this is your proof
k rollout history deployment/app-v1 > /tmp/exam/rollout-history.txt
k rollout history deployment/app-v1 --revision=2      # what changed in one revision
k rollout undo deployment/app-v1                     # back one revision
k rollout undo deployment/app-v1 --to-revision=1     # back to a specific one
k rollout restart deployment/coredns -n kube-system # re-roll pods with no spec change

strategy:
  type: RollingUpdate
  rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
```

`rollout restart` is the workhorse for "make the pods pick up my ConfigMap change" — it appears in the CoreDNS and TLS labs (17, 25, 64). A ConfigMap edit does **not** restart consumers by itself.

4.5 HPA (Labs 5, 29, 40)

```
k autoscale deployment apache-deployment --cpu-percent=50 --min=1 --max=4 -n autoscale
```

The imperative form covers most tasks. You need YAML the moment `behavior` is mentioned — a stabilisation window, for instance, has no flag:

```
apiVersion: autoscaling/v2      # v2, not v1 – v1 has no metrics/behavior blocks
kind: HorizontalPodAutoscaler
metadata: { name: apache-server, namespace: autoscale }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: apache-deployment }
  minReplicas: 1
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 50 }
  behavior:
    scaleDown: { stabilizationWindowSeconds: 30 }
```

HPA READS <UNKNOWN>?

Two causes: metrics-server is not installed, or the target pods have **no CPU requests**. Utilisation is a percentage *of the request* — with no request there is nothing to compute a percentage against. If a task asks for an HPA on a Deployment with no resources set, setting requests is part of the answer.

4.6 Multi-container pods, probes and config

```
# Sidecar sharing a volume (Labs 3, 49, 63)
volumes: [ { name: logs, emptyDir: {} } ]
containers:
- name: app
  volumeMounts: [ { name: logs, mountPath: /var/log } ]
- name: sidecar
  image: busybox:stable
  command: ['sh','-c','tail -F /var/log/app.log'] # -F survives log rotation; -f does not
  volumeMounts: [ { name: logs, mountPath: /var/log } ]

# Probes (Lab 55)
livenessProbe: { httpGet: { path: /, port: 80 }, initialDelaySeconds: 5 }
readinessProbe: { httpGet: { path: /, port: 80 }, initialDelaySeconds: 5 }

# ConfigMap as a volume (Lab 54) – each key becomes a FILE in the directory
volumes: [ { name: cfg, configMap: { name: app-config } } ]
containers:
- volumeMounts: [ { name: cfg, mountPath: /etc/config } ] # -> /etc/config/APP_COLOR
```

LIVENESS VS READINESS, STATED SO YOU NEVER MIX THEM UP

Liveness fails → the container is killed and restarted (you see RESTARTS climbing). **Readiness fails** → the pod is removed from Service endpoints but keeps running (you see 0/1 READY with 0 restarts). That single distinction diagnoses half the "service returns nothing" questions in Part 7.

4.7 Pod Security Standards (Lab 31)

```
k label namespace prod pod-security.kubernetes.io/enforce=restricted
# modes: enforce (reject) | audit (log) | warn (message). levels: privileged | baseline | restricted

# A restricted-compliant pod needs all five of these:
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    seccompProfile: { type: RuntimeDefault }
  containers:
  - securityContext:
    allowPrivilegeEscalation: false
    capabilities: { drop: ["ALL"] }
```

Note the split: `runAsNonRoot` / `seccompProfile` sit at **pod** level, while `allowPrivilegeEscalation` and `capabilities` sit at **container** level. Putting them at the wrong level is the usual rejection reason.

4.8 LimitRange and ResourceQuota (Lab 39)

	Scope	Effect
LimitRange	Per container / pod	Injects defaults into pods created without resources, and rejects pods outside min/max.
ResourceQuota	Per namespace, aggregate	Caps the total across the namespace. Once a quota on CPU/memory exists, every new pod must declare requests and limits or it is rejected.

That interaction is the exam-worthy part: a ResourceQuota alone will break plain `kubectl run`. Pairing it with a LimitRange that supplies defaults is what makes the namespace usable.

Storage

The smallest domain, but it has the most precise failure modes. Storage questions are rarely about writing YAML — they are about understanding **why a PVC will not bind**.

5.1 The binding rules — memorise these five

A PVC binds to a PV only when **all** of the following hold. Lab 61 breaks three of them at once.

#	Rule	What "not matching" looks like
1	storageClassName matches exactly	PV says <code>manual</code> , PVC says <code>fast</code> → Pending forever. Empty string <code>" "</code> means "no class" and is not the same as omitting the field.
2	accessModes are satisfied	PVC asks for <code>ReadOnlyMany</code> , PV only offers <code>ReadWriteOnce</code> → no bind.
3	PV capacity ≥ PVC request	A 1Gi PV can never satisfy a 5Gi claim. The reverse is fine — you just waste space.
4	volumeMode matches	Filesystem vs Block. Rarely tested, instantly fatal.
5	The PV is Available	A <code>Released</code> PV (its old claim was deleted, reclaimPolicy Retain) will not re-bind until the stale <code>spec.claimRef</code> is cleared.

LAB 61'S REAL LESSON: ORDER OF OPERATIONS

PVC spec fields (`storageClassName`, `accessModes`, `resources.requests`) are **immutable**. To fix a wrong PVC you must recreate it — but `pvc-protection` blocks deletion while a pod is using it, so the delete hangs. The sequence is:

```
# 1. pod first 2. then the PVC 3. then recreate both
k delete pod data-consumer -n ns
k delete pvc data-claim -n ns
k apply -f fixed-pvc-and-pod.yaml
k get pvc data-claim -n ns      # Bound
```

5.2 StorageClasses (Labs 14, 27, 48)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"  # quoted string, not a bool
provisioner: rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain

# Flip the default — and REMOVE it from the old one. Two defaults = undefined behaviour.
k patch storageclass local-storage -p \
  '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
k patch storageclass local-path -p \
  '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
k get sc                                     # exactly ONE should say (default)
```

volumeBindingMode	Behaviour
Immediate	PV is provisioned as soon as the PVC is created — possibly on a node the pod cannot use.
WaitForFirstConsumer	Binding is deferred until a pod actually schedules. The PVC sitting Pending is correct, not broken. This is the right answer for local/topology-constrained storage.

EXPECTED-PENDING, NOT BROKEN

Two situations where a Pending PVC is the *correct* end state and you should not "fix" it:

`WaitForFirstConsumer` with no consuming pod yet, and provisioner `kubernetes.io/no-provisioner` with no matching PV (Lab 48). Read the task before you start debugging.

5.3 Manual PV with node affinity (Lab 28)

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: manual-pv }
spec:
  capacity: { storage: 1Gi }
  accessModes: [ ReadWriteOnce ]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""           # must match the PVC's "" exactly
  hostPath: { path: /mnt/data }
  nodeAffinity:                  # pins the volume to one node — REQUIRED for local/hostPath
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - { key: kubernetes.io/hostname, operator: In, values: ["node01"] }
```

Reclaim policies: `Retain` keeps the data and leaves the PV `Released` for manual recovery (this is what makes Lab 1's "reuse the existing PV" scenario work); `Delete` destroys the backing volume with the claim.

5.4 Attaching storage to a pod

```
spec:
  volumes:
    - name: data
      persistentVolumeClaim: { claimName: data-pvc }
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - { name: data, mountPath: /usr/share/nginx/html }
```

StatefulSet volumeClaimTemplates (Lab 33)

A StatefulSet gives each replica its **own** PVC, named `<template>-<statefulset>-<ordinal>` — e.g. `www-web-0`. Note that `volumeClaimTemplates` is a sibling of `template`, not inside it, and that `serviceName` must name a headless Service or the per-pod DNS records never appear.

```
spec:
  serviceName: web-svc           # must be a headless Service (clusterIP: None)
  replicas: 3
  template: { ... }
  volumeClaimTemplates:         # SIBLING of template, not a child
    - metadata: { name: www }
```

```
spec:
  storageClassName: cold
  accessModes: [ ReadWriteOnce ]
  resources: { requests: { storage: 1Gi } }
```

DELETING A STATEFULSET DOES NOT DELETE ITS PVCS.

That is by design — the data outlives the workload. If you recreate the StatefulSet, it re-attaches the existing claims. Clean-up is manual: `k delete pvc -l app=web` .

Services & Networking

6.1 Service types, and the two fields people confuse

Type	Reachable from	Notes
ClusterIP	Inside the cluster	The default. Gets a virtual IP and a DNS name.
NodePort	<code><anyNodeIP>:30000-32767</code>	Also creates a ClusterIP underneath.
LoadBalancer	External LB	Stays <code><pending></code> on a bare-metal exam cluster. That is normal.
Headless (<code>clusterIP: None</code>)	Inside the cluster	No virtual IP. DNS returns one A record <i>per pod</i> . Required for StatefulSets.

PORT VS TARGETPORT VS NODEPORT

`port` = what the Service listens on. `targetPort` = the **container's** port. `nodePort` = the port opened on every node. A wrong `targetPort` is the single most common cause of "the Service exists, has endpoints, and still returns nothing" — it appears as a deliberate bug in Labs 56 and 58.

```
k expose deployment echo -n echo-sound --name=echo-service \
  --type=NodePort --port=8080 --target-port=8080

# Headless service (Lab 16)
spec:
  clusterIP: None
  selector: { app: web }
  ports: [ { port: 80, targetPort: 80, protocol: TCP } ]

# Prove it is headless: one A record per pod, not a single ClusterIP
k run dnstest --rm -it --image=busybox:stable --restart=Never -n relative \
  -- nslookup nodeport-headless.relative.svc.cluster.local
```

6.2 Cluster DNS names — the pattern that unlocks every DNS question

```
<service>.<namespace>.svc.cluster.local      # Service
<pod-ip-with-dashes>.<namespace>.pod.cluster.local
<pod>.<service>.<namespace>.svc.cluster.local  # StatefulSet pod, e.g. web-0.web-svc.ns...
```

Inside a pod, `/etc/resolv.conf` lists search domains, so a bare `web-svc` resolves within the same namespace and `web-svc.other-ns` resolves across namespaces. Custom search domains are set via `spec.dnsConfig` (Lab 34).

```
# Your universal DNS probe — memorise this one-liner
k run dnstest --rm -it --image=busybox:stable --restart=Never -- nslookup <name>
```

6.3 Ingress and Gateway API (Labs 11, 12, 38)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: echo, namespace: echo-sound }
spec:
  rules:
  - host: example.org
    http:
      paths:
      - path: /echo
        pathType: Prefix          # pathType is REQUIRED in networking.k8s.io/v1
        backend:
          service:
            name: echo-service
            port: { number: 8080 }
```

Gateway API splits Ingress into two objects with separate ownership: the **Gateway** (infrastructure — listeners, ports, TLS) and the **HTTPRoute** (application — hostnames, paths, backends). Migrating an Ingress means reading its host, TLS secret and backend, then re-expressing them across the two.

```
kind: Gateway                      # apiVersion: gateway.networking.k8s.io/v1
spec:
  gatewayClassName: nginx-class    # check: k get gatewayclass
  listeners:
  - { name: https, protocol: HTTPS, port: 443, hostname: gateway.web.k8s.local,
      tls: { mode: Terminate, certificateRefs: [ { kind: Secret, name: web-tls } ] } }
  ---
kind: HTTPRoute
spec:
  parentRefs: [ { name: web-gateway } ]      # attaches route to gateway
  hostnames: [ "gateway.web.k8s.local" ]
  rules:
  - matches: [ { path: { type: PathPrefix, value: / } } ]
    backendRefs: [ { name: web-service, port: 80 } ]
```

Read the source before you write: `k describe ingress web` gives you the host, the paths and the backend service/port; `k describe secret web-tls` confirms the TLS secret name. Both labs are graded on reproducing those exactly.

6.4 NetworkPolicy — think in four questions

THE FOUR-QUESTION METHOD

1. **Which pods am I protecting?** → `podSelector` (empty `{}` = all pods in the namespace).
2. **Which direction?** → `policyTypes`: `[Ingress]`, `[Egress]`, or both.
3. **Who is allowed?** → `from:` / `to:` with `podSelector`, `namespaceSelector` or `ipBlock`.
4. **On which ports?** → `ports:` with protocol.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: db-policy, namespace: networking }
spec:
  podSelector: { matchLabels: { role: db } }      # 1. protecting
  policyTypes: [ Ingress ]                       # 2. direction
  ingress:
```

```
- from:
- podSelector: { matchLabels: { role: frontend } } # 3. who
ports:
- { protocol: TCP, port: 3306 } # 4. ports
```

THE FIVE NETWORKPOLICY TRAPS

1. **Policies are additive; there is no deny rule.** You cannot "override" a default-deny — you add allow policies alongside it. Union of all matching policies wins (Lab 58).
2. **Listing a policyType with no rules means deny-all for that direction.** That is how default-deny is written.
3. **An egress policy blocks DNS.** Port 53 UDP *and* TCP to kube-dns must be allowed explicitly, or the pod cannot even resolve the service name it is allowed to reach. This is the most common silent failure.
4. **The YAML dash matters enormously.**
 - namespaceSelector: {...}
 - podSelector: {...} = pods matching **both** (AND).
 - namespaceSelector: {...}
 - podSelector: {...} = **either** (OR). One character changes the security posture.
5. **Nothing is enforced without a policy-capable CNI.** Flannel alone ignores NetworkPolicy entirely; Calico enforces it (Lab 8).

```
# The DNS egress rule you will need again and again
egress:
- to:
  - namespaceSelector: {}
    podSelector: { matchLabels: { k8s-app: kube-dns } }
ports:
- { protocol: UDP, port: 53 }
- { protocol: TCP, port: 53 }
```

6.5 CoreDNS (Labs 25, 64)

CoreDNS is configured by the `coredns` ConfigMap in `kube-system`, which holds a **Corefile**. Two plugin lines account for almost every DNS task:

Corefile line	What it controls
<code>kubernetes cluster.local ...</code>	In-cluster names. Break the zone here and every <code>*.svc.cluster.local</code> lookup fails.
<code>forward . /etc/resolv.conf</code>	External names. Point this at an unroutable address and in-cluster DNS still works while the internet does not.
<code>rewrite name old.example.com svc.ns.svc.cluster.local</code>	Aliases a custom name onto a Service. Must sit before the kubernetes plugin.

THE DIAGNOSTIC SPLIT THAT TELLS YOU WHICH LINE TO LOOK AT

Internal works, external fails → the `forward` line.

Everything fails → the `kubernetes` zone, or CoreDNS is not running at all.

Check the second possibility first: `k get deploy coredns -n kube-system`. Lab 64 opens with CoreDNS scaled to zero replicas — no amount of Corefile debugging helps.

```
k get cm coredns -n kube-system -o yaml           # read the Corefile
k edit cm coredns -n kube-system                  # fix it
k rollout restart deployment coredns -n kube-system # MANDATORY – see below
k rollout status deployment coredns -n kube-system
k logs -n kube-system -l k8s-app=kube-dns --tail=30 # if it crashloops, the Corefile did not parse
```

Why the restart is mandatory: the `reload` plugin polls roughly every 30 seconds, is not always enabled, and refuses to load a Corefile that fails to parse. Restarting is deterministic — and it is a graded step in Lab 25.

6.6 CNI (Lab 8)

Requirements decide the choice. If a task says "must support network policy enforcement", **Flannel is disqualified** — it provides pod networking but no policy engine. Calico provides both.

```
k apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/tigera-operator.yaml
k get pods -n kube-system -o wide           # CNI pods Running on every node
k get nodes                                 # nodes flip to Ready once a CNI is installed
```

A node stuck `NotReady` with `"cni plugin not initialized"` in `kubectl describe node` means exactly this: no CNI. It is a common first task on a fresh exam cluster.

Troubleshooting — the domain that decides your result

Nearly a third of the marks. The good news: broken clusters fail in a small number of recognisable ways. What follows are five playbooks. Between them they cover every troubleshooting lab in this repo — and, in my experience of these scenarios, almost everything the exam asks.

7.0 The universal opening move

Whatever is broken, spend the first 45 seconds here. Do not guess.

```
k get pods -A -o wide | grep -Ev 'Running|Completed'    # everything unhealthy, cluster-wide
k describe pod <pod> -n <ns> | tail -25                # EVENTS are at the bottom – read them
k logs <pod> -n <ns> [-c <container>]                  # current logs
k logs <pod> -n <ns> --previous                          # logs from the CRASHED instance <- CrashLoop
key
k get events -n <ns> --sort-by=.metadata.creationTimestamp | tail -20
```

READ THE STATUS COLUMN — IT NAMES THE PLAYBOOK

Kubernetes tells you which of the five playbooks you are in before you type a second command. **Pending** → §7.1. **ImagePullBackOff** / **CrashLoopBackOff** / **CreateContainerConfigError** → §7.2. **Running** but **unreachable** → §7.3. **Node NotReady** → §7.4. A control-plane pod missing or restarting → §7.5.

7.1 Playbook A — Pod is Pending

Pending means **no node has been chosen**. The scheduler almost always tells you why, in the pod's events.

```
k describe pod <pod> -n <ns> | grep -A10 Events
```

Event message contains	Cause and fix
Insufficient cpu/memory	Requests exceed any node's allocatable. Lower the request (Lab 62: a request of <code>4</code> that was meant to be <code>100m</code>) or free capacity.
didn't match Pod's node affinity/selector	A <code>nodeSelector</code> or affinity references a label no node has. Fix the label or remove the selector.
had taint {...}, that the pod didn't tolerate	Add the toleration, or remove the taint with a trailing <code>-</code> .
pod has unbound immediate PersistentVolumeClaims	Storage problem, not a scheduling one → go to Part 5 §5.1.
No events at all	The most important case. Nothing is even trying to schedule it → the scheduler is down, or the pod names a <code>schedulerName</code> that does not exist. Go to §7.5.

Lab 62 is deliberately two independent causes at once — an impossible CPU request *and* a bogus nodeSelector. Read **every** scheduler message, not just the first. Multi-fault labs are the norm in this repo because they are the norm in the exam.

7.2 Playbook B — Pod will not start

Status	Meaning and first command
ImagePullBackOff ErrImagePull	The image name, tag, or registry is wrong — or the registry needs credentials. <code>k describe pod</code> shows the exact pull error. Lab 56 uses <code>nginx:1.99-fake</code> .
CreateContainerConfigError	A referenced ConfigMap or Secret does not exist. This is not a crash — the kubelet cannot even build the container. <code>describe</code> names the missing object. Create it and the kubelet retries by itself; no pod delete is needed (Lab 57).
CrashLoopBackOff	The container starts, then exits. <code>k logs --previous</code> is the only command that shows you why. Common causes: bad command, missing env var, a liveness probe killing a healthy-but-slow app.
Running but 0/1 READY	Readiness probe failing. The app is alive; the probe is wrong. Check path, port and <code>initialDelaySeconds</code> . Lab 56 probes <code>/healthz</code> , which nginx answers with a 404.
OOMKilled in describe	Memory limit too low. Raise it (Lab 45: 32Mi → 256Mi).
Init:0/1 stuck	The init container is failing, and it blocks everything after it. <code>k logs <pod> -c <init-container></code> . Labs 57 and 63 both hide a fault here.

TWO RULES FOR MULTI-CONTAINER PODS

(1) `kubectl logs` without `-c` on a multi-container pod either errors or silently picks one container. Always name the container. (2) **Container specs are immutable.** You cannot edit a bare pod's image, command or volumeMounts in place — you must delete and recreate it (Labs 61, 63). Inside a *Deployment* you can, because the controller creates a new pod for you.

7.3 Playbook C — Service returns nothing

Work outward from the pod. Four checks, in this order, and stop at the first that fails.

```
# 1. Are the pods actually Ready? A non-Ready pod is never an endpoint.
k get pods -n <ns> -l <selector>

# 2. THE decisive command. Empty endpoints = selector/label mismatch. Populated = a port problem.
k get endpoints <svc> -n <ns>

# 3. Compare the two strings character by character
k get svc <svc> -n <ns> -o jsonpath='{.spec.selector}{"\n"}'
k get pods -n <ns> --show-labels

# 4. targetPort must equal the container's real listening port
k get svc <svc> -n <ns> -o yaml | grep -A4 ports
k get deploy <d> -n <ns> -o jsonpath='{.spec.template.spec.containers[0].ports}'

# 5. Prove it end to end from inside the cluster
k run tmp --rm -it --image=busybox:stable --restart=Never -- wget -qO- http://<svc>.<ns>:80
```

ENDPOINTS IS THE FORK IN THE ROAD

No endpoints → the Service selector does not match the pod labels, or no pod is Ready.

Endpoints present but no response → wrong `targetPort`, the app is not listening, or a NetworkPolicy is dropping the traffic.

Lab 56 packs four bugs into this one flow: bad image, wrong probe path, misspelt selector (`app=webstack` vs `app=web-stack`) and wrong `targetPort`. Fix them layer by layer — do not try to see all four at once.

7.4 Playbook D — Node is NotReady

A NotReady node is nearly always a **kubelet** problem, and the kubelet is a systemd service, not a pod. So `kubecttl` will not fix it — you need to be on the node.

```
k describe node <node> | grep -A15 Conditions # which condition is failing?

# --- on the node itself ---
systemctl status kubelet --no-pager
journalctl -u kubelet -n 50 --no-pager # THE most useful command in this playbook
systemctl restart kubelet
```

journalctl says	Fix
failed to load kubeconfig ... kubelet.conf	Regenerate it on a control-plane node: <code>kubeadm init phase kubeconfig kubelet</code> (Lab 20).
cni plugin not initialized	No CNI installed → Part 6 §6.6.
no space left on device / DiskPressure	Free disk; the kubelet evicts and refuses pods under pressure.
failed to run Kubelet: ... cgroup	Container-runtime/cgroup-driver mismatch. Check <code>/var/lib/kubelet/config.yaml</code> and the runtime config.
Service is inactive (dead)	<code>systemctl enable --now kubelet</code> .

NODE-LEVEL CONFIGURATION LIVES IN TWO FILES

`/var/lib/kubelet/config.yaml` — the kubelet's own config (`maxPods` , `staticPodPath` , eviction thresholds). Lab 69 sets `maxPods: 40` here, then restarts the kubelet and verifies with `k get node <n> -o jsonpath='{.status.capacity.pods}'` .
`/etc/systemd/system/kubelet.service.d/10-kubeadm.conf` — the flags systemd passes in. After editing anything under systemd: `systemctl daemon-reload && systemctl restart kubelet` .

7.5 Playbook E — A control-plane component is down

The highest-value playbook in the guide. Four labs here (15, 22, 60, plus etcd in 73) are variations of one scenario, and the fix is always the same shape: **a bad static pod manifest on disk**.

WHICH SYMPTOM MAPS TO WHICH COMPONENT

Symptom	Component
<code>kubecttl</code> itself fails — connection refused	kube-apiserver (or etcd beneath it)
Pods stay Pending, no scheduling events at all	kube-scheduler
Deployments create no ReplicaSet; nothing scales, endpoints never populate	kube-controller-manager
API server up but every write fails / data is stale	etcd

```
# 1. Look. Static pods are named <component>-<nodename>.
k get pods -n kube-system

# 2. If kubecttl works, read the logs normally
```

```
k logs -n kube-system kube-controller-manager-$(hostname) --tail=20

# 3. If kubectl does NOT work (apiserver/etcd down), drop to the runtime.
# This is the command that saves the etcd and apiserver questions.
crictl ps -a | grep kube-apiserver
crictl logs $(crictl ps -a | grep kube-apiserver | awk '{print $1}' | head -1) 2>&1 | tail -30
journalctl -u kubelet -n 50 --no-pager # kubelet reports why a static pod won't start

# 4. Fix the manifest on disk. NEVER 'kubectl edit' a static pod.
vi /etc/kubernetes/manifests/kube-apiserver.yaml
# typical faults: wrong --etcd-servers endpoint or port
# --kubeconfig pointing at a file that does not exist
# a bad volume hostPath, a typo'd flag, a wrong image tag

# 5. The kubelet reloads the manifest automatically. Force it if impatient:
mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/ && sleep 10 && mv /tmp/kube-apiserver.yaml
/etc/kubernetes/manifests/

# 6. Verify
k get pods -n kube-system
k get nodes
```

FOUR HABITS THAT MAKE THIS PLAYBOOK RELIABLE

1. **Back up before you edit:** `cp /etc/kubernetes/manifests/x.yaml /tmp/x.yaml.bak` . A typo in an apiserver manifest takes the cluster away from you entirely.
2. **Wait 30-60 seconds after saving.** The kubelet's re-sync is not instant. People "fix" a correct manifest twice because they checked too early.
3. **Look for the referenced file existing.** Most of these faults are a path that points at nothing: `controller-manager-missing.conf` , `scheduler-wrong.conf` . Compare against `ls /etc/kubernetes/` .
4. **crictl is your fallback shell.** `crictl ps -a` , `crictl logs <id>` , `crictl pods` . When the API server is down, this is the only way to see what the container actually said.

7.6 Draining a node, and PodDisruptionBudgets (Lab 70)

```
k drain <node> --ignore-daemonsets --delete-emptydir-data
# --ignore-daemonsets is almost always required: DaemonSet pods cannot be evicted.
# --delete-emptydir-data is required when any pod uses an emptyDir volume.
k uncordon <node> # put it back in service when maintenance is done
```

If the drain hangs with *"Cannot evict pod as it would violate the pod's disruption budget"*, a PDB with `maxUnavailable: 0` (or `minAvailable` equal to the replica count) is blocking every eviction. **Do not delete the PDB** — relax it:

```
k patch pdb pdb-app-guard -n ns --type=merge -p '{"spec":{"maxUnavailable":1}}'
```

Then re-run the drain; evictions proceed one pod at a time, which is exactly what a PDB is for.

7.7 Cluster-level inspection

```
k get nodes -o wide
k top nodes ; k top pods -A # needs metrics-server
k get componentstatuses # deprecated but still quick
k cluster-info
k get events -A --sort-by=.metadata.creationTimestamp | tail -30
k api-resources --verbs=list --namespaced -o name # "what exists in this namespace?"
```

PART 8

The command arsenal

If you can reproduce this page from memory, you have the mechanical half of the exam covered. Print it, cover the right column, and drill it.

8.1 Inspect

```
k get pods -A -o wide           # everything, everywhere, with node + IP
k get all -n <ns>              # quick namespace survey
k get pods --show-labels       # label debugging
k get pods -l app=web          # select by label
k get pods --field-selector status.phase=Pending -A
k describe <kind> <name> -n <ns> # events live at the bottom
k get <kind> <name> -o yaml      # the full truth
k get pod <p> -o jsonpath='{.spec.nodeName}{"\n"}'
k get pods -o custom-columns=NAME:.metadata.name,CPU:.spec.containers[*].resources.requests.cpu
k get events -n <ns> --sort-by=.metadata.creationTimestamp
```

8.2 Create and modify

```
k apply -f file.yaml           k delete -f file.yaml
k edit <kind> <name> -n <ns>    # fastest for a one-field change
k scale deployment web --replicas=3
k set image deployment/web nginx=nginx:1.25
k set resources deployment/web --requests=cpu=250m,memory=300Mi
k label node node01 disk=ssd    k label node node01 disk-          # '-' removes
k annotate ns prod owner=team
k taint node node01 key=val:NoSchedule k taint node node01 key=val:NoSchedule-

# Strategic merge patch – merges by key, safest for containers
k patch deployment web -p '{"spec":{"replicas":5}}'
# JSON patch – surgical, needs an exact path; use it to replace one field
k patch svc web-svc --type=json -p='[{"op":"replace","path":"/spec/ports/0/targetPort","value":80}]'
# Merge patch – replaces whole sub-objects
k patch pdb guard --type=merge -p '{"spec":{"maxUnavailable":1}}'
```

8.3 Debug and exec

```
k logs <pod> -c <container> --previous --tail=50
k logs -l app=web --all-containers --tail=20    # by label, across pods
k exec -it <pod> -c <c> -- sh
k exec <pod> -- curl -s --max-time 5 http://web-svc
k run tmp --rm -it --image=busybox:stable --restart=Never -- sh # throwaway shell
k run dnstest --rm -it --image=busybox:stable --restart=Never -- nslookup <name>
k port-forward svc/web 8080:80
k cp <pod>:/path/file ./file
k auth can-i <verb> <resource> -n <ns> --as=<user|system:serviceaccount:ns:sa>
k auth can-i --list -n <ns> --as=<identity>
```

8.4 On the node (no kubectl required)

```
systemctl status kubelet --no-pager      journalctl -u kubelet -n 50 --no-pager
systemctl daemon-reload && systemctl restart kubelet
crictl ps -a                             crictl logs <container-id>
crictl pods                              crictl images
ls /etc/kubernetes/manifests/            # apiserver, etcd, scheduler, controller-manager
cat /var/lib/kubelet/config.yaml          # maxPods, staticPodPath, eviction
kubeadm certs check-expiration            kubeadm certs renew <cert>
kubeadm upgrade plan                     kubeadm init phase kubeconfig kubelet
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates -text | head -20
```

8.5 Contexts and namespaces

```
k config get-contexts                    k config use-context <ctx>
k config current-context
k config set-context --current --namespace=<ns>
k config view --minify                   # just the current context
```

Twenty traps these labs were built to punish

Every item below is a bug deliberately planted in a lab in this repo, or a mistake its solution notes call out. If you internalise nothing else from this guide, internalise this list.

#	Trap	Why it costs you the mark
1	Singular resource in RBAC	<code>resources: ["pod"]</code> matches nothing and errors nowhere. Always plural. Lab 59
2	Wrong apiGroup in a Role	Deployments live in <code>apps</code> , pods in <code>"</code> . A perfect-looking Role that denies everything. 23, 59
3	ClusterRoleBinding when a RoleBinding was wanted	Grants the verb in every namespace and fails the negative check. Lab 23
4	Subject namespace omitted in a RoleBinding	A ServiceAccount subject needs its own <code>namespace:</code> ; it does not inherit. Lab 59
5	Forgetting <code>uncordon</code> after an upgrade or drain	The node stays <code>SchedulingDisabled</code> and the task is marked incomplete. 24, 70
6	Editing a ConfigMap and expecting it to take effect	Consumers keep the old copy until restarted: <code>k rollout restart deployment/<d></code> . 17, 25, 64
7	Restoring etcd over the live data directory	Restore to a <i>new</i> dir and repoint the static pod's hostPath. Lab 73
8	<code>kubectl edit</code> on a static pod	Static pods are files on disk. The mirror pod is read-only and reverts. 67, 15, 22, 60
9	Deleting the object you were told to fix	Labs 61, 70 and 74 explicitly grade you on preserving the PDB, the PV, and the pod's UID. 61, 70, 74
10	Two default StorageClasses	Setting a new default without clearing the old one gives undefined provisioning. Lab 14
11	Trying to edit an immutable PVC field	<code>storageClassName</code> / <code>accessModes</code> / <code>size</code> cannot change. Delete the pod, then the PVC, then recreate. Lab 61
12	Egress NetworkPolicy without a DNS rule	The pod cannot resolve the very service it is permitted to reach. Allow <code>53/UDP</code> and <code>53/TCP</code> . Lab 58
13	The NetworkPolicy dash	Two selectors under one dash = AND. Two dashes = OR. One character, opposite meaning. 42, 51, 58
14	Expecting a policy to deny	NetworkPolicies only allow; they are additive. You cannot override default-deny except by adding allows. Lab 58
15	Wrong <code>targetPort</code>	Service exists, endpoints populate, nothing responds. Check the container's real port. 56, 58
16	Selector / label typo	<code>app=webstack</code> vs <code>app=web-stack</code> → zero endpoints, no error anywhere. Lab 56
17	Requests above limits	Rejected outright. When your calculated request exceeds the limit, go under the limit. 4, 19

18	Guessing a Helm flag instead of finding the value	<code>--skip-crds</code> does not work for argo-cd; <code>--set crds.install=false</code> does. Run <code>helm show values</code> . Lab 2
19	<code>tail -f</code> in a sidecar	Lowercase <code>-f</code> loses the file after rotation. Use <code>-F</code> . Lab 3
20	Only fixing the first bug	Labs 56 (4 bugs), 61 (3), 57/58/62/63/64 (2 each) mirror the exam: keep reading events until you have found them all. 56-64

TURN THIS TABLE INTO A DRILL

Run `scripts/exam-mode.sh start --questions "56 57 58 59 61 62 63 64"` with a 50-minute clock. That is a pure troubleshooting mock covering traps 1, 2, 4, 11-16 and 20 in one sitting. If you can score 66% on that set under time, the exam's troubleshooting domain will not surprise you.

Study plan, lab index and exam day

10.1 A six-week plan

Built around the domain weights, not around the order the labs happen to be numbered in. Assumes roughly 6-8 hours a week. If you have less time, extend the calendar rather than cutting weeks 5 and 6 — the mock exams are where the marks come from.

Week	Focus	Labs to run	Goal for the week
1	Fundamentals & speed	46, 52, 53, 54, 55, 49, 47, 43	Aliases, generators and <code>explain</code> become automatic. Every one of these labs in under 4 minutes, no docs.
2	Workloads & scheduling	3, 4, 5, 7, 10, 18, 19, 29, 30, 32, 39, 40, 44, 74	Resource maths without hesitating; taints vs affinity chosen correctly first time.
3	Storage & networking	1, 14, 26, 27, 28, 33, 48 · 12, 13, 16, 34, 42, 51	Recite the five PVC binding rules. Write a NetworkPolicy with the four-question method.
4	Cluster architecture	23, 41, 50, 65, 66 · 36, 37, 71, 72 · 67, 68, 69, 24, 73	Rehearse etcd backup/restore and the kubeadm upgrade three times each. These are the biggest single blocks of marks on the paper.
5	Troubleshooting	15, 20, 21, 22, 25, 45, 56, 57, 58, 59, 60, 61, 62, 63, 64, 70	Work each one <i>without</i> reading SolutionNotes first. Diagnosis is the skill; the fix is usually trivial once you have named the fault.
6	Mock exams	<code>exam-mode.sh</code> × 3	Three full timed papers with different seeds. Score 75%+ twice before you book.

THE MOCK-EXAM PROTOCOL

```
# Sit a full paper. --safe if you only have a single-node cluster.
scripts/exam-mode.sh start -n 17 -t 120 -s 42
scripts/exam-mode.sh status
scripts/exam-mode.sh grade
scripts/exam-mode.sh cleanup
```

Rules for it to be worth anything: phone away, one kubernetes.io tab only, no notes, no going over time. Afterwards, re-run `validate-question.sh` on every question you lost points on and read that lab's `SolutionNotes.bash` — that is the actual learning, not the sitting. Re-use the same seed (`-s 42`) a week later to measure improvement on identical questions.

10.2 Reading your score

Mock score	What to do
< 50%	Not a timing problem — a knowledge problem. Go back to the weakest domain's chapter and re-run those labs untimed.
50-65%	Usually timing and skipping. Re-sit with strict three-pass discipline (§1.3) and see how much the same knowledge is worth.
66-74%	A pass, but with no margin. Find your two weakest domains and drill only those.
75%+ twice	Book the exam.

10.3 Exam-day checklist

- Set up `alias k`, completion, `$do`, and `~/.vimrc` before task 1.
- Open the kubernetes.io docs tab and leave it open.
- **Run the context-switch command on every single task.**
- Re-read the task after finishing it. Names, namespaces, ports, file paths.
- Flag and skip anything you cannot start in 60 seconds.
- Back up any file under `/etc/kubernetes/manifests/` before editing it.
- After a control-plane edit, wait 30–60 seconds before judging it.
- Uncordon anything you drained.
- Confirm rollouts with `rollout status`, not with `apply` succeeding.
- Keep 15–20 minutes for a verification pass over everything.
- Never leave a task blank — a partial object scores more than nothing.

THE LAST THING TO REMEMBER

You need 66%, not 100%. Three complete, verified tasks are worth more than five half-finished ones. Move fast, verify hard, and let the difficult question go.

10.4 Lab index — all 74 labs

Run any of them with `scripts/run-question.sh N` and grade with `scripts/validate-question.sh N`. Labs marked **Troubleshooting** and **Cluster Arch** carry 55% of the exam's weight between them — prioritise accordingly. Some cluster-architecture labs need control-plane access or a second node; those carry a **# REQUIRES:** marker and are filtered out by `exam-mode.sh --safe`.

#	Lab	Domain	What it drills
1	MariaDB Persistent volume	Storage	Reuse a retained PV: create a matching PVC and wire it into a Deployment
2	ArgoCD	Cluster Arch	Helm template a chart at a pinned version with CRDs suppressed via a chart value
3	Sidecar	Workloads	Add a co-located sidecar sharing an emptyDir log volume
4	Resource Allocation	Workloads	Divide node allocatable evenly across pods and containers
5	HPA	Workloads	HPA with a scale-down stabilization window (needs autoscaling/v2 YAML)
6	CRDs	Cluster Arch	Discover CRDs and read a custom resource schema with kubectl explain
7	PriorityClass	Workloads	Create a PriorityClass relative to existing ones, then patch a Deployment
8	CNI Network Policy	Networking	Install a CNI that also enforces NetworkPolicy
9	Cri Dockerd	Cluster Arch	Install cri-dockerd and set kernel sysctl parameters
10	Taints Tolerations	Workloads	Taint a node and schedule a pod with the matching toleration
11	Gateway API	Networking	Migrate an Ingress (host + TLS) to Gateway API: Gateway + HTTPRoute
12	Ingress	Networking	Expose a Deployment as NodePort and route to it with an Ingress
13	Network Policy	Networking	Choose the least-permissive of three candidate NetworkPolicies
14	Storage Class	Storage	Create a StorageClass, then make it the ONLY default
15	Etcd Fix	Troubleshooting	kube-apiserver down after a migration: fix the etcd endpoint in the static pod
16	NodePort	Networking	NodePort service plus a headless service resolving to pod IPs
17	TLS Config	Networking	Restrict nginx to TLSv1.3 via ConfigMap, then restart to load it
18	Kubectl Patch	Workloads	Raise container limits in place with kubectl patch, leaving requests alone
19	Resource Allocation v2	Workloads	Size requests with (allocatable - overhead) / replicas, capped by limits
20	Kubelet Fix	Troubleshooting	Node NotReady: regenerate a deleted kubelet.conf with kubeadm

21	CrashLoop Fix	Troubleshooting	Diagnose a CrashLooping Deployment from describe and logs
22	Scheduler Fix	Troubleshooting	Pods Pending with no events: kube-scheduler manifest points at a missing kubeconfig
23	RBAC	Cluster Arch	Role + RoleBinding, then a ClusterRole bound per-namespace; verify the negatives
24	Kubeadm Upgrade	Cluster Arch	Full kubeadm patch upgrade: plan, kubeadm, control plane, drain, kubelet, uncordon
25	CoreDNS	Troubleshooting	CoreDNS forward plugin blackholed; add a rewrite rule and reload
26	PVC Pod Storage	Storage	PVC from a StorageClass, mounted into a pod
27	Default StorageClass	Storage	Create a StorageClass and mark it default by annotation
28	Manual PV PVC	Storage	Manual hostPath PV with node affinity + a PVC using storageClassName ""
29	Deployment HPA	Workloads	Deployment with requests/limits plus an HPA
30	Node Affinity	Workloads	Label a node and require it with nodeAffinity
31	Pod Security Restricted	Workloads	Enforce the restricted Pod Security Standard and write a compliant pod
32	Taint Toleration Deploy	Workloads	Taint a node; prove one Deployment tolerates it and another does not
33	Headless StatefulSet	Storage	Headless Service + StatefulSet with volumeClaimTemplates
34	DNS Debugging	Networking	Custom dnsConfig search domain and in-cluster name resolution
35	DNS Test File	Networking	Run nslookup inside a pod and capture results to a file
36	Helm Install	Cluster Arch	helm install with --create-namespace and --set overrides
37	Kustomize	Cluster Arch	Kustomize base + production overlay with a configMapGenerator
38	Gateway API Routes	Networking	Gateway API: one Gateway, path-based HTTPRoute to two services
39	LimitRange ResourceQuota	Workloads	LimitRange defaults interacting with a ResourceQuota
40	Resource Consumer HPA	Workloads	Resource-consumer Deployment driving a CPU-based HPA
41	RBAC App Admin	Cluster Arch	ServiceAccount + Role across two API groups + a pod that uses the SA
42	Tiered NetworkPolicies	Networking	Three-tier NetworkPolicies: web to api to db
43	Rolling Update Rollback	Workloads	RollingUpdate strategy, image update, rollout history, rollback
44	PriorityClass AntiAffinity	Workloads	PriorityClasses with preemption plus required pod anti-affinity

45	Troubleshoot Deployment	Troubleshooting	Three planted bugs: containerPort, memory limit, liveness probe port
46	Pod With Label	Workloads	Minimal pod with a specific label
47	DaemonSet HostPath	Workloads	DaemonSet with a hostPath mount tolerating all taints
48	StorageClass No Provisioner	Storage	no-provisioner StorageClass; a Pending PVC is the correct end state
49	Multi Container EmptyDir	Workloads	Two containers sharing an emptyDir volume
50	RBAC Default Namespace	Cluster Arch	ServiceAccount, Role and RoleBinding in the default namespace
51	NetworkPolicy Port	Networking	NetworkPolicy restricted to a single TCP port
52	Deployment NodePort	Networking	Deployment exposed by a NodePort Service
53	Pod Resources	Workloads	Pod-level CPU and memory requests and limits
54	ConfigMap Volume	Workloads	Mount a ConfigMap as a volume (keys become files)
55	Liveness Readiness	Workloads	Liveness and readiness probes on the same pod
56	Broken App Stack	Troubleshooting	FOUR bugs: bad image, wrong probe path, selector typo, wrong targetPort
57	CrashLoop Missing Config	Troubleshooting	CreateContainerConfigError: a missing ConfigMap and a missing Secret
58	NetworkPolicy Debug	Troubleshooting	Wrong targetPort plus default-deny needing ingress, egress AND DNS allows
59	RBAC Debug	Troubleshooting	RBAC with a singular resource, missing verbs and a wrong subject namespace
60	Controller Manager Fix	Troubleshooting	No ReplicaSets created: kube-controller-manager kubeconfig path is wrong
61	PVC Binding Fix	Troubleshooting	PVC/PV mismatch in three fields; immutable spec forces a careful delete order
62	Stuck Rollout	Troubleshooting	Two independent scheduling faults: impossible CPU request and a bogus nodeSelector
63	Sidecar Volume Fix	Troubleshooting	Sidecar sharing broken by a readOnly mount and a mismatched mountPath
64	CoreDNS Debug	Troubleshooting	Cluster-wide DNS outage: CoreDNS scaled to zero AND a corrupted cluster zone
65	ClusterRole Aggregation	Cluster Arch	Aggregated ClusterRoles selected by label, bound to a ServiceAccount
66	User CSR Kubeconfig	Cluster Arch	Onboard a user end to end: key, CSR, approval, RoleBinding, kubeconfig
67	Static Pod	Cluster Arch	Static pod on the control-plane node and its mirror pod behaviour
68	Cert Renewal	Cluster Arch	kubeadm certs check-expiration and renewing a single certificate

69	Kubelet MaxPods	Cluster Arch	Set maxPods in the kubelet config file and verify node capacity
70	PDB Drain	Cluster Arch	A PDB blocks drain; relax it without deleting it, then drain and leave cordoned
71	Kustomize Advanced	Cluster Arch	Advanced Kustomize: namePrefix, JSON6902 patch, secretGenerator, annotations
72	Helm Lifecycle	Cluster Arch	Helm lifecycle: install, upgrade, history, rollback
73	EtcD Backup Restore	Cluster Arch	etcd snapshot save, status, restore to a new data dir, repoint the static pod
74	Manual Scheduling	Workloads	Manual scheduling with spec.nodeName and the Binding API

KEEP THE GUIDE ALIVE

This guide was generated from the labs as they stand today. When you add a new `Question-N-Topic/` directory, remember to bump the `seq 1 N` bound in `scripts/validate-question.sh` and `scripts/cleanup-question.sh`, or `all` will silently skip it — and add a row here so the index stays honest.

CKA Field Guide · generated from the PhilaniMhlongo/CKA lab repository · domain weights and exam logistics should be re-checked against the current CNCF exam page before you book.